IT5437 Computer Vision - Assignment 01 -Intensity Transformations and Neighbourhood Filtering

Weerarathna T.M.P. | **Index No: 258843A** | GitHub: https://github.com/PathmikaW/cv-assignment01

Q1 - Intensity Transforms on Runway Image

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q1_intensity_transforms.py

Three LUT-based point transforms applied to the runway grayscale image using cv.LUT().

(a) Gamma correction: gamma = 0.5

s = r^0.5, r in [0,1]. Gamma less than 1 expands the dark pixel range, brightening shadow regions.

```
gamma = 0.5
t = np.array([(i/255.0)**(gamma)*255 for i in np.arange(0,256)]).astype(np.uint8)
g_05 = cv.LUT(f, t)
```

(b) Gamma correction: gamma = 2.0

s = r^2.0. Gamma greater than 1 compresses low intensities, darkening the image.

```
gamma = 2.0
t = np.array([(i/255.0)**(gamma)*255 for i in np.arange(0,256)]).astype(np.uint8)
g_2 = cv.LUT(f, t)
```

(c) Contrast Stretching: r1=0.2, r2=0.8

Piecewise linear: output = 0 if r < r1, linear ramp if r1 <= r <= r2, 255 if r > r2. LUT built with np.linspace and np.concatenate.

```
t1 = np.linspace(0,   0,   p1).astype(np.uint8)          # r < r1  → 0
t2 = np.linspace(0, 255, p2 - p1 + 1).astype(np.uint8)   # linear ramp
t3 = np.linspace(255, 255, 255 - p2).astype(np.uint8)    # r > r2  → 255
t = np.concatenate((t1, t2, t3)).astype(np.uint8)
g_cs = cv.LUT(f, t)
```

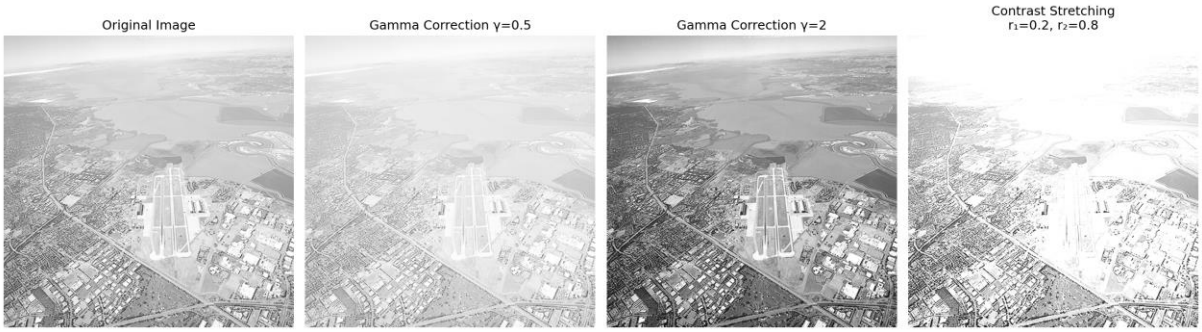

Figure 1. Original, gamma=0.5, gamma=2.0, contrast-stretched (r1=0.2, r2=0.8).

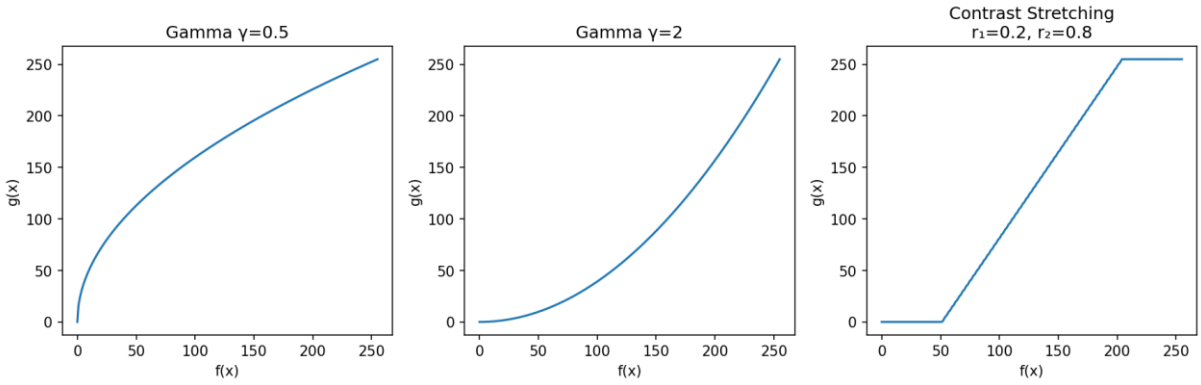

Figure 2. Transform function plots for each operation.

Q2 - Gamma Correction in L*a*b* Colour Space

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q2_lab_gamma.py

Image converted from BGR to L*a*b* using cv.COLOR_BGR2Lab. Gamma correction with gamma=0.5 applied only to the L* (luminance) channel via LUT, then merged back with the original a* and b* channels. This brightens the image without shifting colour or hue. Gamma value used: 0.5.

```
lab = cv.cvtColor(img, cv.COLOR_BGR2Lab)
L, a, b = cv.split(lab)
gamma = 0.5
t = np.array([(i/255.0)**(gamma)*255 for i in np.arange(0,256)]).astype(np.uint8)
L_corrected = cv.LUT(L, t)
lab_corrected = cv.merge([L_corrected, a, b])
img_corrected = cv.cvtColor(lab_corrected, cv.COLOR_Lab2BGR)
```

(a) Original and corrected images

(b) Histograms of original L* and corrected L*

Figure 3. Original vs corrected (gamma=0.5 on L only).*

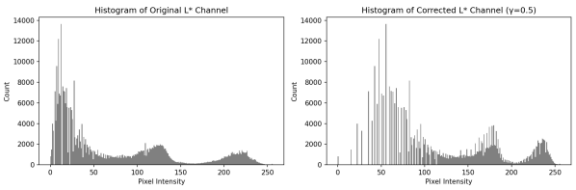

Figure 4. L histograms before and after correction.*

Q3 - Custom Histogram Equalization on Runway Image

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q3_hist_equalization.py

Custom equalize_hist() implements the formula: s_k = (L-1)/MN * CDF(k), without any built-in equalization function.

```python
def equalize_hist(img):
    L = 256
    M, N = img.shape
    hist, _ = np.histogram(img.flatten(), 256, [0, 256])    # Compute histogram
    cdf = hist.cumsum() # Compute CDF
    t = np.round((L - 1) / (M * N) * cdf).astype(np.uint8)
    g = t[img] # Apply transform
    return g, hist, t
```

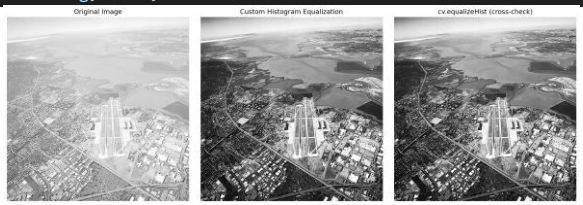

Figure 5. Original, custom equalised, cv.equalizeHist

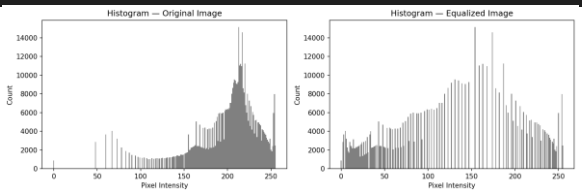

Figure 6. Histograms before and after equalization.

Q4 - Otsu Thresholding and Selective Equalization

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q4_otsu_foreground.py

(a) Otsu Thresholding

Otsu's method finds the threshold that minimises intra-class intensity variance. Resulting threshold value = **101.0** Pixels above 101 form the foreground mask covering the woman and the room interior.

```python
ret, mask = cv.threshold(gray, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)
```

(b) Histogram Equalization on Foreground Only

CDF computed from foreground pixels only (mask == 255). LUT applied only to foreground; background unchanged. Hidden features revealed: furniture outlines, wall texture, and shadow detail in the dark room interior become clearly visible.

```python
foreground_pixels = gray[mask == 255]
L = 256
n_fg = foreground_pixels.size # Build equalization LUT from foreground pixels only
hist, _ = np.histogram(foreground_pixels, 256, [0, 256])
cdf = hist.cumsum()
t = np.round((L - 1) / n_fg * cdf).astype(np.uint8)
g_eq = gray.copy() # Apply LUT to full image, then restore only foreground
g_eq[mask == 255] = t[gray[mask == 255]]
```

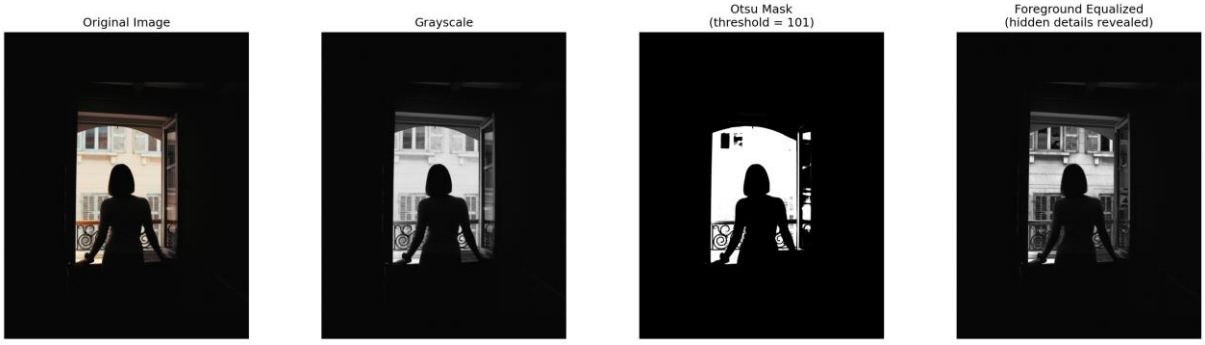

Figure 7. Original, grayscale, Otsu mask (threshold=101), foreground-equalized result.

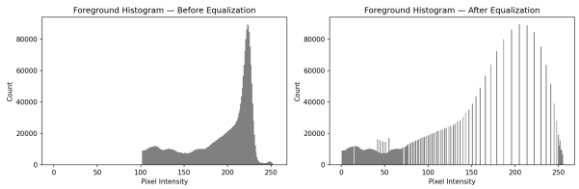

Figure 8. Foreground histograms before and after equalization.

Figure 9. Terminal output showing Otsu threshold = 101.

Q5 - Gaussian Filtering (sigma = 2)

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q5_gaussian_filtering.py

(a) Normalised 5x5 Gaussian kernel using NumPy

Built with np.meshgrid. G = exp(-(x^2+y^2)/(2*sigma^2)), normalised so coefficients sum to 1. Centre value = 0.0632. Kernel is symmetric.

```
ax_range = np.arange(-(ksize//2), ksize//2 + 1)
x, y = np.meshgrid(ax_range, ax_range)
G5 = np.exp(-(x**2 + y**2) / (2 * sigma**2))
G5 = G5 / G5.sum()   # normalise so kernel sums to 1
```

(b) 51x51 kernel as 3D surface

Larger kernel plotted with ax.plot_surface(). Confirms the smooth, rotationally symmetric bell shape.

(c) Manual kernel applied with cv.filter2D

(d) cv.GaussianBlur comparison

cv.filter2D(f, -1, G5) and cv.GaussianBlur(f, (5,5), sigmaX=2) produce visually identical results, confirming the manual kernel is correct.

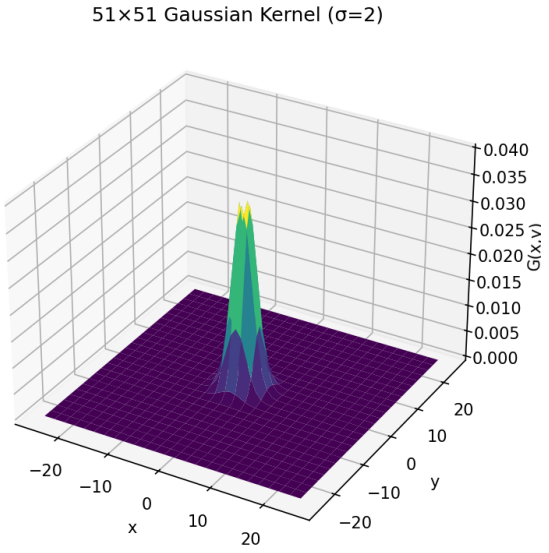

Figure 10. 51x51 Gaussian kernel as 3D surface.

Figure 11. Original, manual filter, cv.GaussianBlur.

Figure 12. Terminal output showing 5x5 kernel coefficient values.

Q6 - Derivative of Gaussian (sigma = 2)

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q6_derivative_of_gaussian.py

(a) Proof of first-order partial derivatives

Given: G(x,y) = 1/(2*pi*sigma^2) * exp(-(x^2+y^2)/(2*sigma^2))

Applying the chain rule to the exponential term:

dG/dx = 1/(2*pi*sigma^2) * exp(-(x^2+y^2)/(2*sigma^2)) * (-2x / 2*sigma^2) = -(x/sigma^2) * G(x,y)

By symmetry (replacing x with y): dG/dy = -(y/sigma^2) * G(x,y).

(b) 5x5 DoG kernels for x and y directions

each normalised by their absolute sum. DoGx is antisymmetric in x with a zero centre column; DoGy is antisymmetric in y.

```
G = np.exp(-(x**2 + y**2) / (2 * sigma**2))
DoGx = (-x / sigma**2) * G
DoGy = (-y / sigma**2) * G
DoGx = DoGx / np.abs(DoGx).sum()
DoGy = DoGy / np.abs(DoGy).sum()
```

(c) 51x51 DoG-x kernel as 3D surface

Plotted with plot_surface(). Shows the characteristic bipolar shape with positive and negative lobes.

(d) Image gradients using DoG kernels

cv.filter2D(f, cv.CV_64F, DoGx) and DoGy applied to the runway image. CV_64F preserves negative gradient values. Magnitude = sqrt(gx^2 + gy^2).

(e) Sobel comparison and discussion

cv.Sobel(f, cv.CV_64F, 1, 0, ksize=5) applied for comparison. Both methods detect the same edges. DoG uses the true Gaussian derivative and is smoother and more isotropic. Sobel uses integer-approximate coefficients and is faster but slightly less smooth and more sensitive to noise at large kernel sizes.

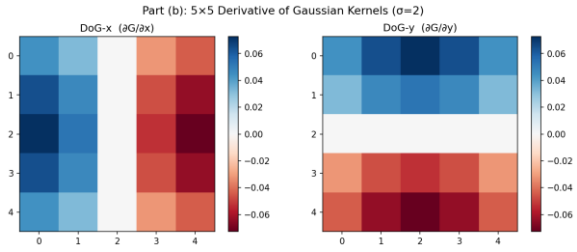

Figure 13. 5x5 DoG-x and DoG-y kernel heatmaps.

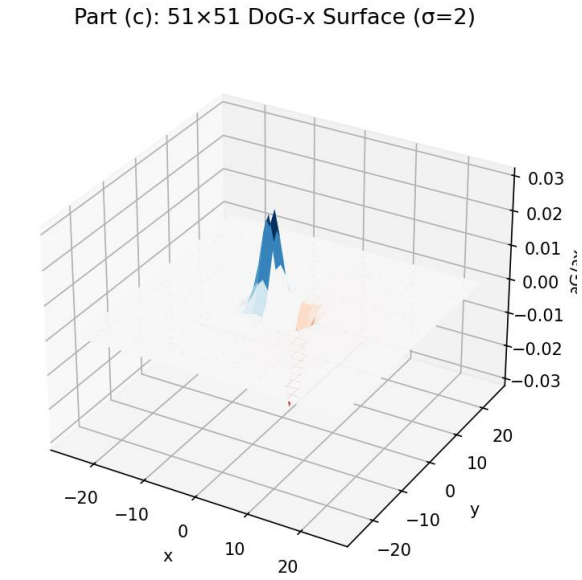

Figure 14. 51x51 DoG-x kernel as 3D surface.

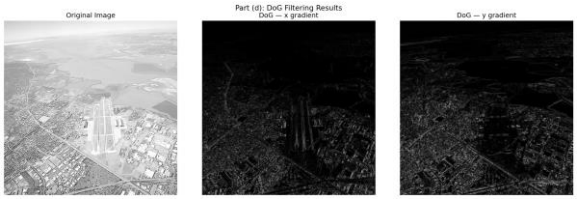

Figure 15. DoG x and y gradients applied to runway image.

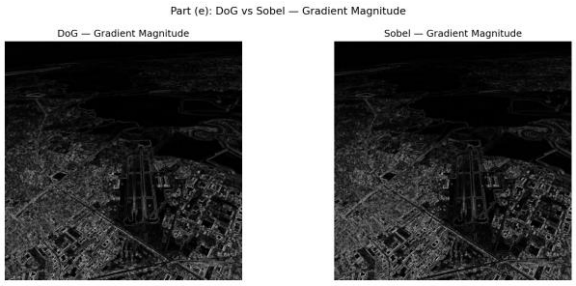

Figure 16. DoG gradient magnitude vs Sobel gradient magnitude.

```
(cv-assignment01) PS D:\MSc\Sem 03\IT5437-Computer Vision\Assignment1\cv-assignment01> uv run python src/q6_derivative_of_gaussian.py
Part (a) — Derivation:
  ∂G/∂x = -(x/σ²) * G(x,y)
  ∂G/∂y = -(y/σ²) * G(x,y)
Part (b) — 5x5 DoG-x kernel:
[[ 0.0441  0.0321  0.     -0.0321 -0.0441]
 [ 0.0642  0.0467  0.     -0.0467 -0.0642]
 [ 0.0728  0.0529  0.     -0.0529 -0.0728]
 [ 0.0642  0.0467  0.     -0.0467 -0.0642]
 [ 0.0441  0.0321  0.     -0.0321 -0.0441]]

Part (b) — 5x5 DoG-y kernel:
[[ 0.0441  0.0642  0.0728  0.0642  0.0441]
 [ 0.0321  0.0467  0.0529  0.0467  0.0321]
 [ 0.      0.      0.      0.      0.    ]
 [-0.0321 -0.0467 -0.0529 -0.0467 -0.0321]
 [-0.0441 -0.0642 -0.0728 -0.0642 -0.0441]]
```

Figure 17. Terminal output showing 5x5 DoG-x and DoG-y kernel values.

Q7 - Image Zoom with Interpolation (s in range (0, 10])

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q7_image_zoom.py

zoom_image(img, s, method) maps each output pixel (i, j) back to source coordinates: src_y = i/s, src_x = j/s.

(a) Nearest-neighbour interpolation

Source coordinates rounded to the nearest integer. Fast but produces blocky pixel artefacts at high zoom factors.

```
            sy = min(int(np.round(src_y)), H - 1)
            sx = min(int(np.round(src_x)), W - 1)
            output[i, j] = img[sy, sx]
```

(b) Bilinear interpolation

Weighted average of the 4 surrounding pixels using fractional offsets dx and dy.

```
            top    = (1 - dx) * img[y0, x0] + dx * img[y0, x1]
            bottom = (1 - dx) * img[y1, x0] + dx * img[y1, x1]
            output[i, j] = np.clip((1 - dy) * top + dy * bottom, 0, 255)
```

Normalised SSD = sum((zoomed - original)^2) / N computed against the large originals. im01 (s=4): NN SSD = 137.21, Bilinear SSD = 115.66. taylor (s=5): NN SSD = 239.49, Bilinear SSD = 225.55. Bilinear consistently achieves lower SSD, confirming better reconstruction quality.

Figure 18. im01: original, small input, NN zoom, bilinear zoom (s=4).

Figure 19. taylor: original, small input, NN zoom, bilinear zoom (s=5).

Figure 20. im02 zoom results (s=4).

Figure 21. im03 zoom results (s=4).

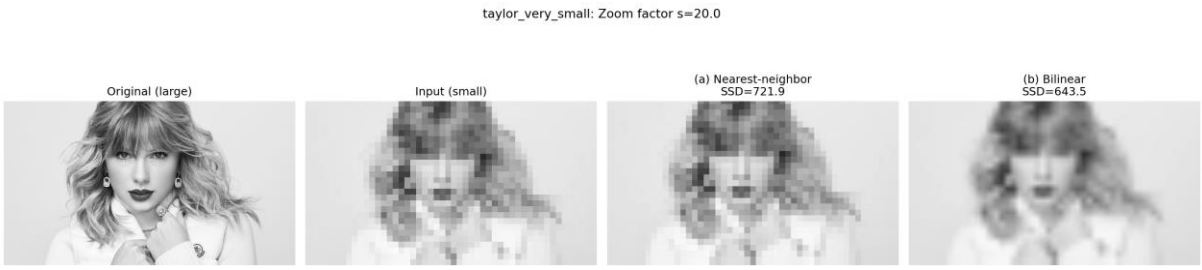

Figure 22. taylor very small input zoom results (s=20, included for completeness).

```
(cv-assignment01) PS D:\MSc\Sem 03\IT5437-Computer Vision\Assignment1\cv-assignment01> uv run python src/q7_image_zoom.py

im01small: small=(270, 480), large=(1080, 1920), zoom factor s=4.00
  Normalized SSD — Nearest-neighbor: 256.82
  Normalized SSD — Bilinear:         201.45

im02small: small=(300, 480), large=(1200, 1920), zoom factor s=4.00
  Normalized SSD — Nearest-neighbor: 65.42
  Normalized SSD — Bilinear:         49.56

im03small: small=(365, 600), large=(1459, 2400), zoom factor s=4.00
  Normalized SSD — Nearest-neighbor: 129.58
  Normalized SSD — Bilinear:         98.34

taylor_small: small=(112, 200), large=(560, 1000), zoom factor s=5.00
  Normalized SSD — Nearest-neighbor: 319.31
  Normalized SSD — Bilinear:         285.64

taylor_very_small: small=(28, 50), large=(560, 1000), zoom factor s=20.00
  Normalized SSD — Nearest-neighbor: 721.89
  Normalized SSD — Bilinear:         643.50
```

Figure 23. Terminal output showing normalised SSD values for all image pairs.

Q8 - Salt and Pepper Noise Filtering (emma.jpg, prob=0.05)

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q8_noise_filtering.py

(a) Gaussian smoothing: cv.GaussianBlur(noisy, (5,5), 1)

Computes a weighted average over the neighbourhood. Extreme salt and pepper values (0 and 255) still influence the weighted mean, so residual artefacts remain and edges are blurred. MSE vs original = 127.37.

```
g_gaussian = cv.GaussianBlur(noisy, (5, 5), 1)
```

(b) Median filtering: cv.medianBlur(noisy, 5)

Replaces each pixel with the sorted median of its neighbourhood. Salt and pepper outliers occupy a small fraction of positions in the sorted list and are never selected as the median, so they are completely eliminated. Edges are preserved because the output is always a real local pixel value. MSE vs original = 54.75, which is less than half the Gaussian error. Median filtering is clearly superior for salt and pepper noise.

```
g_median = cv.medianBlur(noisy, 5)
```

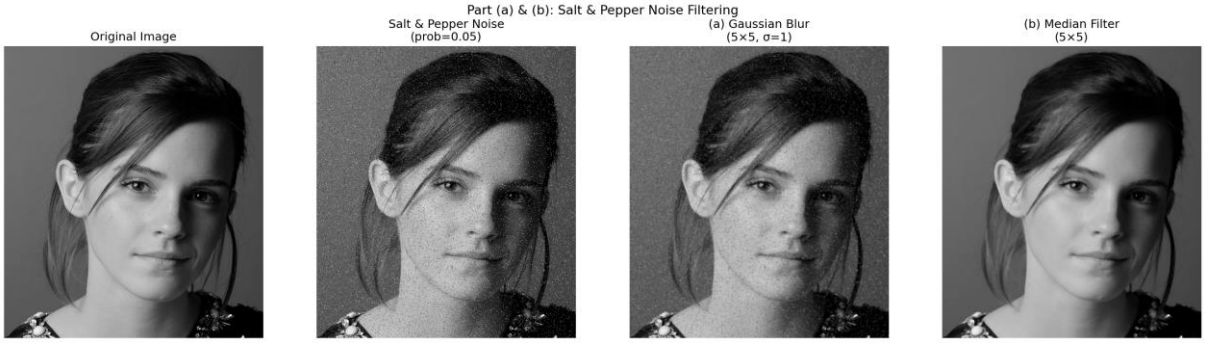

Figure 24. Original, salt and pepper noisy, Gaussian filtered (MSE=127), Median filtered (MSE=55).

```
(cv-assignment01) PS D:\MSc\Sem 03\IT5437-Computer Vision\Assignment1\cv-assignment01> uv run python src/q8_noise_filtering.py
MSE — Noisy vs Original:    1020.45
MSE — Gaussian vs Original: 127.37
MSE — Median vs Original:   54.75
```

Figure 25. Terminal output showing MSE values.

Q9 - Image Sharpening (daisy.jpg)

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q9_sharpening.py

Unsharp masking: sharp = f + alpha * (f - GaussianBlur(f)), with alpha=1.5 and a 5x5 Gaussian blur (sigma=2).

```
blur = cv.GaussianBlur(f, (5, 5), 2)
mask = f.astype(np.float64) - blur.astype(np.float64)
alpha = 1.5
sharp = np.clip(f.astype(np.float64) + alpha * mask, 0, 255).astype(np.uint8)
```

The term (f - blur) isolates high-frequency detail such as edges and fine texture. Scaling by alpha and adding back amplifies those structures. Observed effects: petal edges are noticeably crisper, fine surface texture is more pronounced, and the overall tonal balance is preserved. Setting alpha too large would introduce halo artefacts at strong edges; alpha=1.5 gives a good balance.

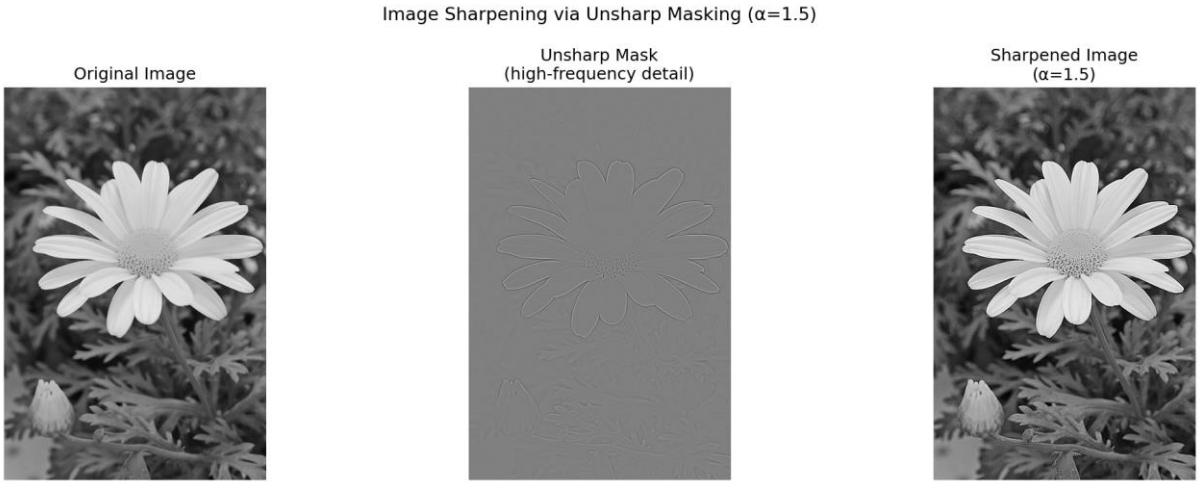

Figure 26. Original daisy, unsharp mask detail, sharpened image with alpha=1.5.

Q10 - Bilateral Filtering (einstein.png, d=9, sigma_s=10, sigma_r=25)

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q10_bilateral_filter.py

Combined weight: w = exp(-dist^2 / (2*sigma_s^2)) * exp(-delta_i^2 / (2*sigma_r^2)). The spatial term penalises distance; the range term penalises intensity difference, naturally preserving edges.

(a) Manual bilateral filter implementation

```
            spatial_weights = np.exp(-(kx**2 + ky**2) / (2 * sigma_s**2))


            intensity_diff = patch - img_pad[i+pad, j+pad]
            range_weights = np.exp(-(intensity_diff**2) / (2 * sigma_r**2))
            # Combined weight
            weights = spatial_weights * range_weights
            output[i, j] = np.sum(weights * patch) / np.sum(weights)
```

(b) cv.GaussianBlur (c) cv.bilateralFilter (d) Comparison

Gaussian blur applies uniform spatial averaging, smoothing noise but smearing all edges. OpenCV bilateral filter preserves edges strongly while smoothing flat regions. The manual bilateral result visually matches the OpenCV result, confirming correct implementation. Key insight: across an edge the intensity difference is large, so the range Gaussian weight drops close to zero and edge pixels are effectively excluded from the average, unlike the Gaussian which weights them equally.

```
g_gaussian = cv.GaussianBlur(f, (d, d), sigma_s)
```

(c)

```
g_bilateral_cv = cv.bilateralFilter(f, d, sigma_r, sigma_s)
```

(d)


```
g_bilateral_manual = bilateral_filter(f, d, sigma_s, sigma_r)
```

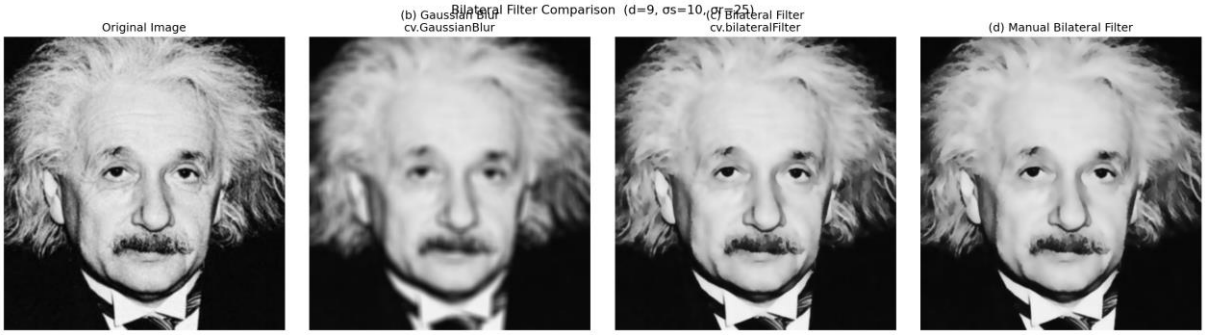

Figure 27. Original, Gaussian blur, cv.bilateralFilter, manual bilateral filter.

Q11 - Spatial Filtering and Frequency Response

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q11_frequency_response.py

(a) Convolution Theorem

The Convolution Theorem states that spatial convolution is equivalent to pointwise multiplication in the frequency domain: G(u,v) = F(u,v) * H(u,v), where F, G, H are the 2D DFTs of f, g, h. H(u,v) is the frequency response of the filter and scales each frequency component of the image independently.

(b) Frequency domain behaviour of each filter

Box filter: low-pass but its sharp rectangular spatial boundary creates strong sidelobes in the frequency domain, causing ringing artefacts in the output. `box = np.ones((9, 9), dtype=np.float64) / 81`

Gaussian filter: also low-pass; its Fourier transform is itself a Gaussian, smooth and monotonically decreasing with no sidelobes and no ringing.

```
sigma = 2
k = 9
ax_range = np.arange(-(k//2), k//2 + 1)
x, y = np.meshgrid(ax_range, ax_range)
gauss = np.exp(-(x**2 + y**2) / (2 * sigma**2))
gauss = gauss / gauss.sum()
```

Laplacian filter: high-pass; amplifies edges and high-frequency content, suppresses DC. Used for edge detection, not noise reduction.

```
laplacian = np.array([[0,  1, 0],
                      [1, -4, 1],
                      [0,  1, 0]], dtype=np.float64)
```

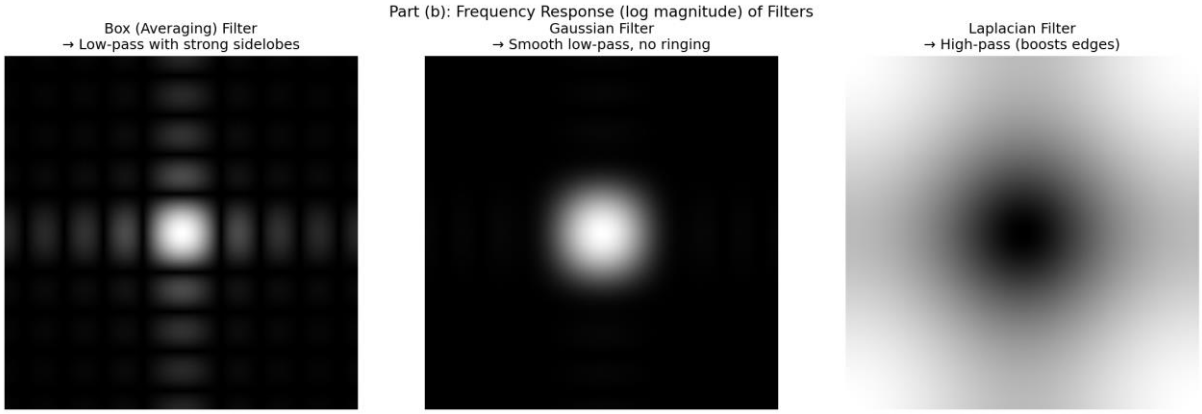

Figure 28. Log-magnitude FFT of the box filter, Gaussian filter, and Laplacian filter.

(c) Why Gaussian filtering avoids ringing

Ringing arises from the Gibbs phenomenon: an abrupt cutoff in the frequency domain produces oscillatory artefacts around edges in the spatial output. The Gaussian function has the unique property that its Fourier transform is also a Gaussian, rolling off smoothly to zero with no discontinuities, so no ringing is produced.

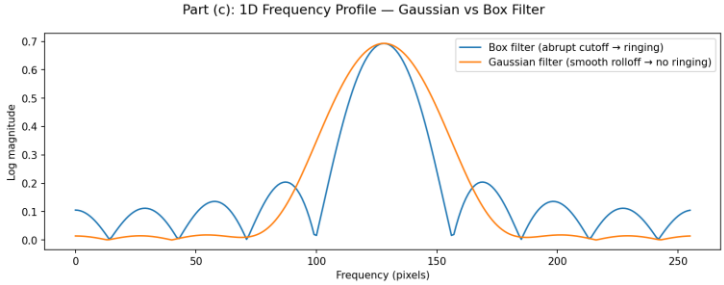

Figure 29. 1D frequency profiles: box filter with abrupt cutoff vs Gaussian with smooth rolloff.

(d) Most suitable filter for high-frequency noise

The Gaussian filter is most suitable. In the spatial domain it suppresses noise via smooth neighbourhood averaging with a bell-shaped weighting. In the frequency domain it attenuates high frequencies, where noise concentrates, smoothly and without ringing. The box filter also attenuates high frequencies but introduces ringing artefacts. The Laplacian amplifies high frequencies and would worsen the noise.

```
f = cv.imread('Assignment/a1images/emma.jpg', cv.IMREAD_GRAYSCALE)

rng = np.random.default_rng(42)
noise = rng.normal(0, 25, f.shape).astype(np.float64)
f_noisy = np.clip(f.astype(np.float64) + noise, 0, 255).astype(np.uint8)
g_box   = cv.filter2D(f_noisy, -1, box.astype(np.float32))
g_gauss = cv.GaussianBlur(f_noisy, (9, 9), sigma)
g_lap   = cv.filter2D(f_noisy, -1, laplacian.astype(np.float32))
```

Figure 30. Filters applied to a noisy image. Gaussian gives the best noise reduction.

Q12 - Homomorphic Filtering

Source: https://github.com/PathmikaW/cv-assignment01/blob/main/src/q12_homomorphic.py

(a) Significance of the multiplicative model

f(x,y) = i(x,y) * r(x,y), where illumination i varies slowly over large spatial regions (low frequency) and reflectance r captures fine surface detail (high frequency). Non-uniform lighting causes i to dominate, masking the detail encoded in r.

(b) Log transform separates illumination and reflectance

log(f) = log(i) + log(r). Multiplication becomes addition, enabling a linear frequency-domain filter to independently attenuate the illumination component and boost the reflectance component.

(c) Homomorphic filtering algorithm

Step 1: z = log(1 + f)

Step 2: Z = FFT(z), shift to centre

Step 3: Apply high-emphasis filter: H(u,v) = (gamma_H - gamma_L) * (1 - exp(-D^2 / (2*sigma^2))) + gamma_L, with gamma_L=0.25, gamma_H=2.0, sigma=0.1. This attenuates low-frequency illumination and boosts high-frequency reflectance.

Step 4: g_log = IFFT(H * Z)

Step 5: g = exp(g_log) - 1, normalised to [0, 255

(d) Comparison with histogram equalization

Histogram equalization stretches global contrast uniformly and can over-enhance already-bright regions. Homomorphic filtering targets the illumination component specifically, normalising the lighting gradient while preserving local reflectance contrast and giving a more natural result. Homomorphic filtering is preferred when a strong illumination gradient dominates the image. Histogram equalization is preferred when simple global contrast enhancement is the goal.

(e) Results and observations

Applied to highlights_and_shadows.jpg. The bright window highlight is reduced and shadow detail in darker regions becomes visible. The result looks more natural than histogram equalization, which over-brightens already well-lit areas. A minor halo artefact is visible at the brightest boundaries where the high-emphasis filter transition is steepest.

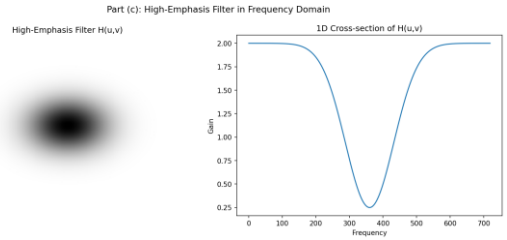

Figure 31. High-emphasis filter H(u,v) and 1D cross-section.

Figure 32. Original, homomorphic result, histogram equalization.